01 / RESEARCH QUESTION

Recognizing unseen household objects

Can a frozen DINOv2 encoder improve classification of bottles, cups and pens when the physical objects in the test set have never appeared in training?

A comparison on the original Assignment 1 split

The selected DINOv2 head classified **56 of 60** test photographs correctly, compared with **50 of 60** for the frozen ResNet18 baseline. Macro-F1 changed from 0.826 to 0.933 (+0.107). Both scores refer to exactly the same three held-out objects.



Bottle / training example



Cup / training example



Pen / training example

What the application returns

A user supplies one photograph. The page shows the predicted class, three model scores, CPU inference time and the four nearest original training images. Those examples give a way to inspect the result even when the label is wrong.

How to interpret the evidence

The 60 test photographs cover only one new design per class. A better score therefore supports the new pipeline on this collection; it cannot establish performance across all household objects. A second camera and more held-out designs would be a stronger test.

02 / DATA AND SPLIT

Keeping physical objects separate

A change of viewpoint does not create a new object. The unit of separation here is the bottle, cup or pen itself.

Category	Objects	Train photos	Validation	Test
Bottle	7	100	20	20
Cup	7	100	20	20
Pen	7	100	20	20
Total	21	300	60	60

Collection and matching

The A1 report records an iPhone 15 Pro collection on 7 October 2026: five viewpoints on four backgrounds per object. The surfaces included a tablecloth, linoleum and ceramic. Lighting conditions were not logged. Each class contains seven objects and 140 original photographs.

The A2 run read the saved A1 manifest rather than making another random split. For every image it corrected EXIF orientation, converted to RGB and matched the SHA-256 digest of its dimensions and pixels. All 420 digests were unique. The 60 test digests also matched the saved A1 prediction file.

What changes across the two training conditions

The original condition uses the 300 training images once per full-batch update. The mirror condition adds one horizontally flipped view per training image, giving 600 feature vectors. All validation and test inputs remain original. No generated scenes or synthetic objects are present: the share of generated originals is 0%.

Object folders can contain Cyrillic letters in different Unicode forms. Normalizing names for lookup preserves the original object assignments; hash verification independently confirms the image contents.

03 / MODERN METHOD

From photo to visual evidence

DINOv2 learns a reusable image representation before seeing any of this project's class labels. Here only the final three-class linear layer is fitted to household photographs.

1 RGB input	2 ViT-S/14	3 Two outputs
Fit and pad to 224^2	384-D CLS vector	Class and neighbors

How the representation is learned

During pretraining, a student transformer learns from a teacher's targets for different views of images; the model also learns to predict masked patch representations. This encourages shared cues across views without hand-labeling every pretraining image. The original DINO method and its teacher–student idea are described in [3]; the DINOv2 training procedure is documented in [1].

How one image becomes a class

The app preserves the full image inside a 224 × 224 canvas, filling the margins with a near-mean color. ImageNet channel normalization precedes the frozen ViT-S/14. The encoder processes 16 × 16 patches of 14 × 14 pixels and a class token. The normalized 384-component class-token vector goes into a learned weight matrix with one column per class. Softmax converts the three scores into relative probabilities.

Why a separate retrieval branch helps

The gallery compares the same unit-length vector with each of the 300 original training vectors. Their dot product is cosine similarity. The top four are selected across all labels, so disagreement stays visible. Neighbors are examples of visual resemblance; the linear weights produce the decision. No similar test image is eligible for the gallery.

04 / EXPERIMENT

Training and ablation

The ablation asks a single question: does including a mirror view of each training photo change generalization when the head, optimizer and split stay fixed?

Both heads start at zero and use 180 full-batch gradient updates on cross entropy, learning rate 5 and an L2 gradient of 0.0005 × weights. The encoder is frozen. The earliest epoch with maximum validation macro-F1 wins within each condition. The higher validation score selects the deployed head; an exact tie favors the original condition. Test labels do not enter either choice.

Training condition	Vectors	Best epoch	Val macro-F1	Test macro-F1
Original only	300	3	0.9162	0.933
Original + mirror	600	3	0.9166	0.933

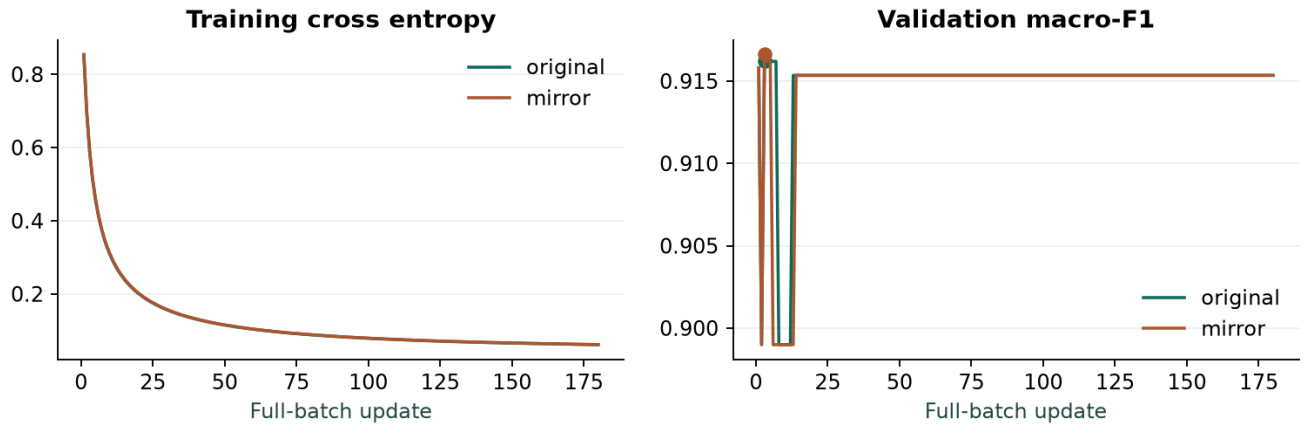


Figure 1. Same optimizer and update count. Dots mark the first validation maximum for each run.

Mirroring changed validation macro-F1 by +0.0004 and test macro-F1 by +0.000. The validation rule selected **mirror**. A small or zero difference is informative on this split: a flip adds another view, but it does not add a new physical design.

A1 used different preprocessing and a different head-training schedule. The baseline comparison therefore concerns two complete pipelines; only the mirror experiment isolates an individual change.

05 / HELD-OUT EVALUATION

What changed on the test set

All models are scored on the same 60 originals. Accuracy counts correct images. Macro-F1 gives bottle, cup and pen equal weight.

Model	Correct / 60	Accuracy	Macro-F1
A1 pixel logistic regression	29	0.483	0.458
A1 ResNet18	50	0.833	0.826
DINOv2 selected	56	0.933	0.933

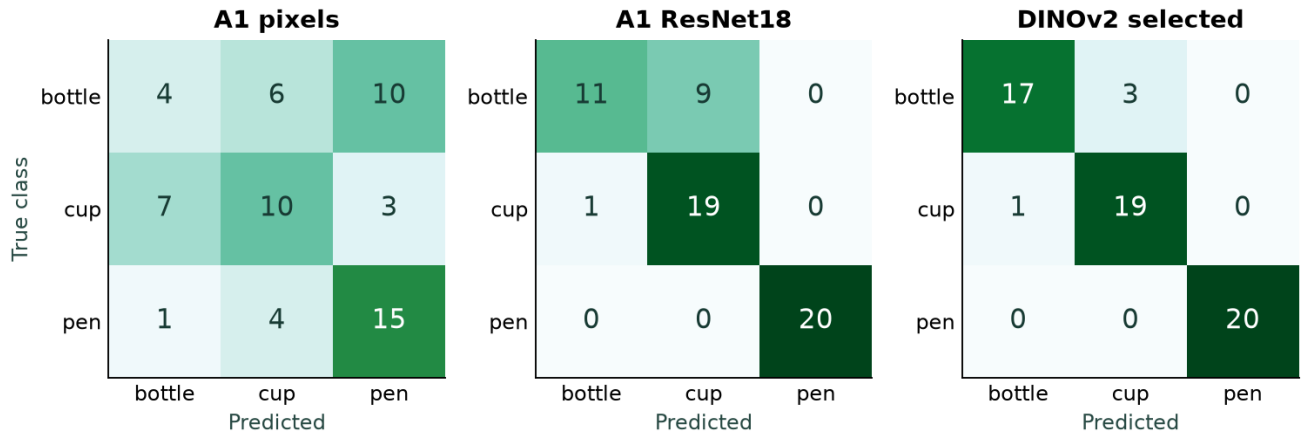


Figure 2. Counts, not percentages. Read each row as 20 photographs of a single real test object.

A paired comparison

The A1 ResNet18 mislabeled nine bottle photos as cups and one cup as a bottle. DINOv2 corrected 8 of those ten errors and introduced 2 error(s) on images that ResNet18 classified correctly. The selected model's class recalls were bottle: 17/20, cup: 19/20, pen: 20/20.

The baseline scores were recomputed from archived per-image predictions rather than retraining A1. The ResNet18 score refers to the saved epoch-12 run. No bounding boxes or masks were collected: detection mAP and segmentation IoU do not evaluate this task.

06 / EXPLAINABILITY

Reading a prediction

In each strip, the first tile is the query and the next four are original training images in decreasing cosine similarity. The query never enters the retrieval index.

Correct prediction / IMG_6180.JPG



True class: bottle. Predicted: bottle. Held-out original.

Nearest labels: 4 bottle. The top match scores 0.636 in cosine similarity. The upright query and all four matches have elongated bodies and visible caps. The earlier ResNet18 predicted cup.

Failure case / IMG_6182.JPG



True class: bottle. Predicted: cup. Held-out original.

Nearest labels: 2 cup, 2 bottle. The top match scores 0.501 in cosine similarity. In this sideways view the nearest image shows a transparent glass. Two neighbors are cups and two are bottles; that ambiguity is visible even though the head returns one label.

Runtime on the demo machine

Over 60 prepared 224 × 224 PNG queries, mean server inference was 43.3 ms (median 42.4 ms) on an AMD Ryzen 5 7535HS, four CPU threads. Timing includes decode, preprocessing, encoder, head and retrieval; it excludes model startup and the browser. Full-resolution JPEG uploads may take longer. The app shows the current request and a rolling mean.

07 / REPRODUCTION AND ETHICS

Running the same experiment

The repository includes the trained head, the training-only index, held-out examples, split evidence and instructions for a local browser demo.

Launch and reproduce

Use Python 3.12 and the pinned packages in requirements.txt. In the repository directory, run the commands below. The model setup verifies the SHA-256 of the pinned ONNX export [2]. After setup, the application runs offline at http://127.0.0.1:7860.

```
python -m pip install -r requirements.txt
python setup_model.py
python app.py
```

For training, download the original A1 Dataset folder, keep its class/object/file layout, then run **python train.py PATH_TO_DATASET**. Look in results for per-image predictions, learning curves and metrics. The archives in evidence identify the original baseline and split.

Limits and responsible use

The app always chooses one of three labels; scores are uncalibrated and cannot reject an unknown object. Photographs from one person, camera and set of surfaces can reward background shortcuts. Collect new object designs, rooms and phones before broader use. A nearest neighbor may expose a private detail: keep the repository access limited, review the images before sharing, and get permission for any identifiable people or spaces. Uploaded files are processed in memory. The video uses synthetic narration; the dataset contains real photographs.

Sources and materials

1 Oquab et al. DINOv2: Learning Robust Visual Features without Supervision

2 DINOv2-small ONNX export and pinned revision

3 Neurohive. FAIR developed DINO architecture for self-supervised image analysis

4 Original A1 dataset on Google Drive

5 Assignment 2 repository and run guide